

풀링과 Global Average Pooling

CNN · 블록 9

앞 블록에서 남긴 것

```
(3, 32, 32) → Conv2d(3, 16, 3, padding=1) → (16, 32, 32)
```

- 채널은 3에서 16으로 **늘었음**
- 면적은 32×32 **그대로**

숫자 개수는 3,072에서 **16,384**로 늘었음

이대로 한 층 더 쌓으면

```
(16, 32, 32) → Conv2d(16, 32, 3, padding=1) → (32, 32, 32)
```

32,768개

층을 쌓을수록 다루는 숫자가 계속 커짐

오늘의 질문

작게 만드는 것은

항상 이득인가?

먼저 손으로

4×4 격자를 2×2로 줄여야 한다면

3	1	0	4
2	8	5	1
0	2	7	3
6	1	2	2

무엇을 남기고 무엇을 버릴 것인가?

풀링은 고르는 동시에 버린다

2×2 창, stride 2 · 창끼리 겹치지 않음

입력 4×4

3	1	0	4
2	8	5	1
0	2	7	3
6	1	2	2

칸 16개



출력 2×2

8	5
6	7

칸 4개

같이 일어난 일

창마다 값 하나만 남고

나머지 12개는 사라짐

8이 어느 칸에 있었는지도

출력만 봐서는 알 수 없음

되돌릴 수 없는 연산임

풀링이 바꾸는 것 — 표본 뽑는 방식, 해상도, 다음 층의 계산량, 남는 정보

풀링이 보장하는 것 — 없음. 뒤에서 하나씩 확인함

풀링이 하는 일

- 겹치지 않는 창마다 값 **하나**를 뽑음
- 나머지는 **사라짐**
- 되돌릴 수 없음

고르는 것과 버리는 것이 같은 동작임

코드로는 두 줄

```
nn.MaxPool2d(kernel_size=2, stride=2)  
nn.AvgPool2d(kernel_size=2, stride=2)
```

- `stride` 를 안 적으면 `kernel_size` 와 같게 잡힘
- 배울 값이 없어서 **파라미터 0개**

넣어 보면

입력	max	avg	
3 1 0 4			
2 8 5 1	8 5	3.50 2.50	
0 2 7 3	6 7	2.25 3.50	
6 1 2 2			

같은 입력, 다른 결과

자주 듣는 말

"max가 average보다

분류에는 낫다"

정말 그런가?

max와 average는 서로 다른 것을 못 본다

max가 구별 못 하는 두 창

	max	avg
9 1 1 1	9	3.0
9 9 9 9	9	9.0

max 값이 똑같이 9 — 두 창을 같은 것으로 만들

average가 구별 못 하는 두 창

	max	avg
4 4 4 4	4	4.0
8 0 8 0	8	4.0

average 값이 똑같이 4 — 두 창을 같은 것으로 만들

max는 세기만 남기고 얼마나 퍼져 있는지를 버림. average는 퍼진 정도를 남기고 뽕족한 값을 눌러 버림
어느 쪽이 분류에 항상 낫다고 말할 수 없음. 무엇을 버려도 되는지가 문제에 따라 다름

둘은 서로 다른 것을 못 봄

	남기는 것	버리는 것
max	가장 센 반응	얼마나 퍼져 있나
average	전체적인 세기	뾰족한 반응

그래서 할 수 있는 말

- 둘은 **다른 정보를 버림**
- 어느 쪽이 항상 낫다고 말할 수 없음

“ 어떤 데이터·어떤 구조에서 어느 쪽이 나왔다는 보고는 있음. 그건 그 조건에서의 결과임 ”

또 자주 듣는 말

"풀링을 하면 위치가
조금 달라져도 같아진다"

조건을 붙이지 않으면 어떻게 되는지 보기

"풀링을 하면 위치가 조금 달라져도 같아진다"

조건을 붙이지 않으면 참이 아니다

원래 자리

0	0	0	0
0	9	0	0
0	0	0	0
0	0	0	0

max pool 2×2 →

9	0
0	0

기준

창 안에서 한 칸

0	0	0	0
9	0	0	0
0	0	0	0
0	0	0	0

max pool 2×2 →

9	0
0	0

출력 같음

창을 넘어 한 칸

0	0	0	0
0	0	9	0
0	0	0	0
0	0	0	0

max pool 2×2 →

0	9
0	0

출력 달라짐

같은 창 안에서 움직였을 때만 출력이 같아짐. 창 경계를 넘으면 출력이 그대로 따라 움직임
즉 풀링은 이동 불변을 주지 않음. 창 크기만큼의 자리 차이를 뭉겔 뿐임

일반화가 좋아지는지, overfitting이 줄어드는지는 이 그림으로는 알 수 없음

확인된 것

- 같은 창 안에서 움직였을 때만 출력이 같음
- 창 경계를 넘으면 출력이 그대로 따라 움직임

폴링은 창 크기만큼의 자리 차이를 뒀을 뿐임
이동 불변을 주는 것이 아님

자주 붙는 세 문장, 다시 보기

들리는 말	실제
위치 변화에 강해짐	창 안일 때만. 보장 아님
일반화가 좋아짐	데이터·구조에 따라 다름
overfitting이 줄어듦	줄어들 수도, 아닐 수도

“ 풀링은 이 셋 중 어느 것도 보장하지 않음 ”

같은 문장이라도 지위가 다르다

계산으로 확인되는 것 · 실험으로 확인되는 것 · 설명일 뿐인 것

계산

- 크기가 1/4 이 된다
- 파라미터가 0 개다

종이와 연필로 확인 가능

실험

- 이 데이터에서
- 정확도가 얼마 나왔다

조건을 밝히면 재현 가능

설명

- 일반화가 좋아진다
- 위치 변화에 강해진다
- overfitting 이 준다

관찰에 나중에 붙인 말

풀링은 오른쪽 칸의 어느 것도 보장하지 않는다

그래도 쓴다. 다만 쓰는 이유가 왼쪽 칸에 있다

새 문장을 만나면 물어보라 — 이걸 계산인가, 실험인가, 설명인가

그러면

폴링에 대해 확실히

말할 수 있는 것은 무엇인가?

확실히 말할 수 있는 것은 크기와 계산량뿐이다

2×2 · stride 2 풀링

입력 shape	원소 수	출력 shape	원소 수
(16, 32, 32)	16,384	(16, 16, 16)	4,096
(32, 16, 16)	8,192	(32, 8, 8)	2,048

1 / 4

면적이 가로·세로 각각
절반이 되므로

"절반"이 아님

학습되는 값

0개

규칙이 고정돼 있어 배울 것이 없음

나누어떨어지지 않으면 가장자리가 버려짐

7×7 → 3×3

8×8 → 4×4

9×9 → 4×4

9에서도 4가 나옴 — 마지막 한 줄은 창을 못 채워서 버려짐

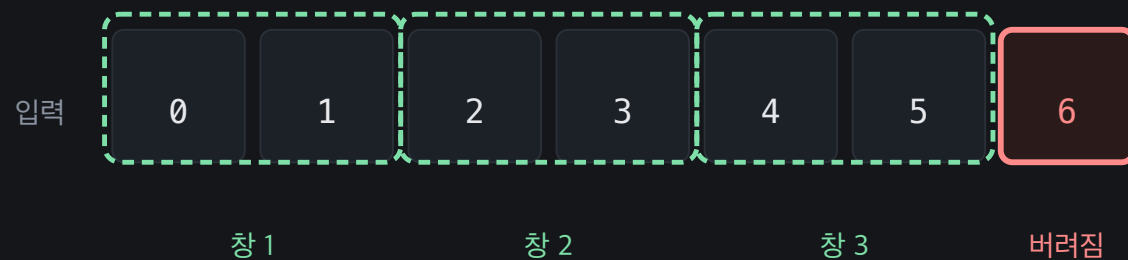
여기까지는 계산으로 확인되는 사실. 성능이 좋아진다는 말은 여기 없음

계산으로 확인되는 것만

원소 수	약 1/4 (절반이 아님)
배우는 값	0개
다음 층 입력	작아짐
나뉘떨어지지 않으면	가장자리가 버려짐

7칸을 2칸씩 묶으면 한 칸이 남는다

남은 칸은 어느 창에도 안 들어가고 그대로 버려진다



기본 (내림)

$$\text{floor}(7 \div 2) = 3$$

창 3개 · 6번 칸은 안 쓰임

PyTorch 기본값

ceil_mode=True (올림)

$$\text{ceil}(7 \div 2) = 4$$

창 4개 · 마지막 창은 반만 채움

버리지 않는 대신 반쪽 창이 생김

홀수 크기가 이어지면 버려지는 줄이 총마다 쌓인다

그래서 실무에서 짝수 크기를 맞춰 두는 경우가 많다

가장자리가 버려지는 자리

7×7	→	3×3	마지막 한 줄이 창을 못 채움
8×8	→	4×4	
9×9	→	4×4	9인데도 4

작은 입력에서 이 손실이 상대적으로 큼

다음 질문

창을 특징 지도 크기만큼 키우면

무슨 일이 일어나는가?

Global Average Pooling — 창을 끝까지 키운 것

채널 한 장을 통째로 평균 내서 숫자 하나로 만든다



무엇이 남고 무엇이 사라지나

남는 것 — 이 채널이 이미지 전체에서
평균적으로 얼마나 반응했나

사라지는 것 — 어디서 반응했나

$8 \times 8 = 64$ 칸이 숫자 1개가 됨

위치를 완전히 버리는 선택임

그게 맞는 문제도, 아닌 문제도 있음

입력 H·W가 8이든 4든 16이든 결과는 (N, 32) — 그래서 뒤에 오는 Linear의 입력 크기가 고정됨

코드와 결과

```
gap = nn.AdaptiveAvgPool2d(1)
gap(torch.randn(4, 32, 8, 8)).shape
```

```
torch.Size([4, 32, 1, 1])
```

```
torch.allclose(gap(x).flatten(1), x.mean(dim=(2, 3))) # True
```

채널마다 평균 하나. 그게 전부임

GAP이 버리는 것

- $8 \times 8 = 64$ 칸이 숫자 1개가 됨
- 어디서 반응했는지가 남지 않음

“ 위치를 완전히 버리는 선택임. 그게 맞는 문제도, 아닌 문제도 있음 ”

마지막 질문

같은 몸통에 머리를

두 가지로 붙이면?

$(N, 32, 8, 8) \rightarrow ? \rightarrow (N, 10)$

머리 부분을 두 가지로 만들어 본다

같은 몸통 (N, 32, 8, 8) 뒤에 붙는다

Flatten head

Flatten()

(N, 32, 8, 8) → (N, 2048)

Linear(2048, 10)

20,490개

어느 자리에서 반응했는지가
가중치에 남아 있음

VS

GAP head

AdaptiveAvgPool2d(1)

(N, 32, 8, 8) → (N, 32)

Linear(32, 10)

330개

자리 정보는 남지 않음
채널별 평균 세기만 남음

확실한 것 — 머리 부분 파라미터가 62배 차이 남

확실하지 않은 것 — 그래서 어느 쪽이 더 잘 맞는지. 데이터와 몸통에 따라 달라짐

파라미터가 적으면 overfitting이 준다는 것도 조건부 가설이지 규칙이 아님

여기서 결론을 내지 않음

확실한 것

머리 부분 파라미터가 20,490개와 330개 — 62배 차이

확실하지 않은 것

어느 쪽이 더 잘 맞는가

“ 파라미터가 적으면 overfitting이 준다는 것도 조건부 가설임 ”

오늘 정리

풀링	고르는 동시에 버림
max · avg	서로 다른 것을 못 봄
이동	창 안일 때만 같아짐
확실한 것	크기 · 계산량 · 파라미터 0
GAP	자리를 완전히 버림

다음 질문

Conv	채널을 바꿈
Pool	면적을 바꿈

부품은 다 모였음

이 부품들을 이어 붙이면

코드가 몇 줄이 되는가?

CNN 모델 구현과 summary

CNN · 블록 10

손에 있는 부품

부품	바꾸는 것	안 바꾸는 것
Conv2d	채널	(padding=10이면) 면적
ReLU	—	전부 그대로
MaxPool2d	면적	채널
AdaptiveAvgPool2d(1)	면적을 1×1로	채널
Flatten	축 개수	값의 개수
Linear	길이	—

오늘의 질문

그림의 각 부분은
코드의 어느 줄인가?

오늘 하는 일

- 완성된 코드를 읽음
- 층마다 (N, C, H, W) 를 적어 내려감
- 코드는 그대로 두고 입력 크기만 바꿔 일부러 깨뜨림

“ 새로 타이핑하지 않음. 읽고 따라가는 시간임

”

오늘 읽을 코드 — 앞쪽

```
features = nn.Sequential(  
    nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(),  
    nn.Conv2d(32, 32, kernel_size=3, padding=1), nn.ReLU(),  
    nn.MaxPool2d(2),  
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),  
    nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.ReLU(),  
    nn.MaxPool2d(2),  
)
```

채널이 3 → 32 → 32 → 64 → 64 로 감

오늘 읽을 코드 — 뒤쪽

```
flat_model = nn.Sequential(  
    features,  
    nn.Flatten(),  
    nn.Linear(64 * 8 * 8, 10),  
)
```

64 * 8 * 8 — 이 숫자가 오늘의 주인공임

그림의 각 부분은 코드의 어느 줄인가

(C, H, W) · 배치 축은 뺀음



면적을 바꾸는 것은 Pool, 채널을 바꾸는 것은 Conv. ReLU는 둘 다 안 바꿈

코드가 두 덩어리인 이유

	하는 일
features	그림에서 쓸 만한 것을 뽑음
뒤쪽 분류기	뽑힌 것을 10개 점수로 바꿈

앞은 그대로 두고 **뒤만 같아 끼울 수 있음**

다음 질문

층을 지날 때마다

shape은 무엇이 되는가?

같이 채워 보기

Input	(64, 3, 32, 32)
Conv + ReLU	(64, __, __, __)
Conv + ReLU	(64, __, __, __)
Pool	(64, __, __, __)
Conv + ReLU	(64, __, __, __)
Conv + ReLU	(64, __, __, __)
Pool	(64, __, __, __)
Flatten	(64, ----)
Linear	(64, --)

층마다 shape 을 한 줄씩 적어 내려간다

채널을 바꾸는 줄과 면적을 바꾸는 줄이 따로 있다

층	나온 shape	바뀐 것
입력	3 x 32 x 32	
Conv2d(3→32)	32 x 32 x 32	채널
Conv2d(32→32)	32 x 32 x 32	채널
MaxPool2d(2)	32 x 16 x 16	면적
Conv2d(32→64)	64 x 16 x 16	채널
Conv2d(64→64)	64 x 16 x 16	채널
MaxPool2d(2)	64 x 8 x 8	면적
GAP	64 x 1 x 1	면적
Linear(64→10)	10	길이

Conv 는 채널만 · Pool 과 GAP 은 면적만 · ReLU 는 아무것도 안 바꾼다

ReLU 다음 줄에서 shape 이 달라져 있으면 잘못 읽은 것

Shape Ledger — 층마다 적어 내려간다

batch_size = 64 · CIFAR-10 입력

층	(N, C, H, W)	N	C	H·W	바뀐 것
Input	(64, 3, 32, 32)	64	3	32×32	—
Conv + ReLU	(64, 32, 32, 32)	64	32	32×32	채널
Conv + ReLU	(64, 32, 32, 32)	64	32	32×32	값만
Pool	(64, 32, 16, 16)	64	32	16×16	면적
Conv + ReLU	(64, 64, 16, 16)	64	64	16×16	채널
Conv + ReLU	(64, 64, 16, 16)	64	64	16×16	값만
Pool	(64, 64, 8, 8)	64	64	8×8	면적
Flatten	(64, 4096)	64	—	—	축이 사라짐
Linear	(64, 10)	64	—	—	10개로

끝까지 안 변함

N은 한 번도 안 바뀜 — 모델은 한 장에 대해 하는 일을 배치의 모든 장에 똑같이 함
Flatten을 지나는 순간 C와 H·W를 따로 말할 수 없게 됨. 여기가 되돌리기 어려운 지점임

- **N 은 끝까지 안 바뀜**

모델은 한 장에 하는 일을 배치의 모든 장에 똑같이 함

- 면적은 Pool에서만, 채널은 Conv에서만 바뀜

Flatten을 지나는 순간

어느 채널의 어느 자리인지를

더는 말할 수 없음

코드에 적힌 숫자 하나

```
nn.Linear(64 * 8 * 8, 10)
```

이 8 은 어디서 왔는가?

4096은 어디서 왔는가

$$64 \times 8 \times 8 = 4096$$

채널 × 세로 × 가로
Flatten이 세 축을 한 줄로 이어 붙임

64

두 번째 블록 마지막 Conv의 out_channels

8

32 → Pool → 16 → Pool → 8

8

가로도 같은 경로

이 숫자를 손으로 적어 두면

```
nn.Linear(64*8*8, 10)
```

64도 8도 앞쪽 설정에서 따라 나온 값임

앞을 고치면 이 줄이 같이 안 따라옴

그래서 잘 깨짐

입력이 32×32라는 것도 여기 숨어 있음

64×64를 넣으면 8이 아니라 16이 됨

세 숫자의 출처

	어디서	
64	마지막 Conv의 <code>out_channels</code>	내가 정함
8	32 → Pool → 16 → Pool → 8	따라 나옴
8	가로도 같은 경로	따라 나옴

입력이 32×32라는 사실도 이 줄에 숨어 있음

실험

코드는 한 글자도 안 고침

입력만 (1, 3, 32, 32) 에서 (1, 3, 64, 64) 로 바꾸면

어디서 무엇이 터질까?

먼저 예측해 보기

- 에러가 나는가, 안 나는가
- 난다면 어느 줄에서
- 메시지에 어떤 숫자가 찍힐까

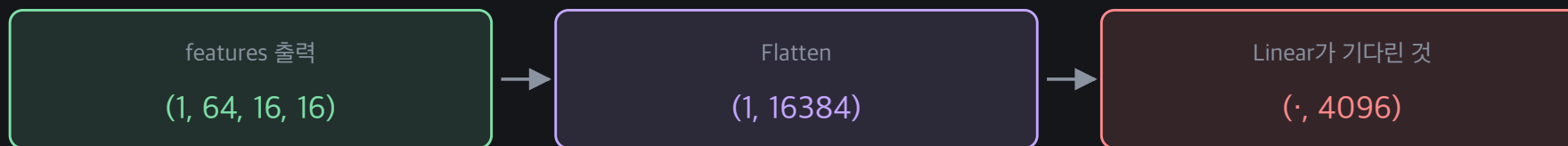
Conv 줄에서 날까, Linear 줄에서 날까?

코드는 그대로 두고 입력만 64×64로 바꿔 본다

Linear에 적어 둔 64*8*8은 손도 대지 않는다

`flat_model(torch.zeros(1, 3, 32, 32))` → (1, 10) 통과

`flat_model(torch.zeros(1, 3, 64, 64))` → ?



16384 ≠ 4096

RuntimeError: mat1 and mat2 shapes cannot be multiplied (1x16384 and 4096x10)

앞의 숫자가 실제로 흘러온 크기

뒤의 숫자가 기다리고 있던 크기

에러가 난 곳은 Linear지만 원인은 그 줄에 없음 — 입력 크기가 바뀐 것임

코드 어디에도 32×32라고 적혀 있지 않은데 32×32에서만 둘

이 메시지 읽는 법

```
mat1 and mat2 shapes cannot be multiplied (1x16384 and 4096x10)
```

1x16384	실제로 흘러온 것
4096x10	기다리고 있던 것

에러가 난 줄과 원인이 있는 줄이 다름

고치는 방법 두 가지

1. 더미 입력으로 재어서 씀

```
with torch.no_grad():  
    n_flat = features(torch.zeros(1, 3, 32, 32)).flatten(1).shape[1]  
nn.Linear(n_flat, 10)
```

2. 뒤쪽을 면적에 안 매이게 바꿈

```
nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(64, 10)
```

1번은 **그 입력 크기에서만** 다시 맞춘 것임

같은 입력 세 크기로 넣어 보면

입력 (2, 3, s, s)	Flatten head	GAP head
32×32	(2, 10)	(2, 10)
48×48	깨짐	(2, 10)
64×64	깨짐	(2, 10)

GAP은 면적이 얼마든 채널 수 **64개**로 요약함

“ 대신 어느 자리에서 나온 값인지는 남지 않음 ”

마지막 질문

이 모델은 값을 몇 개 배우고

어디에 몰려 있는가?

층별 표 — GAP head 모델

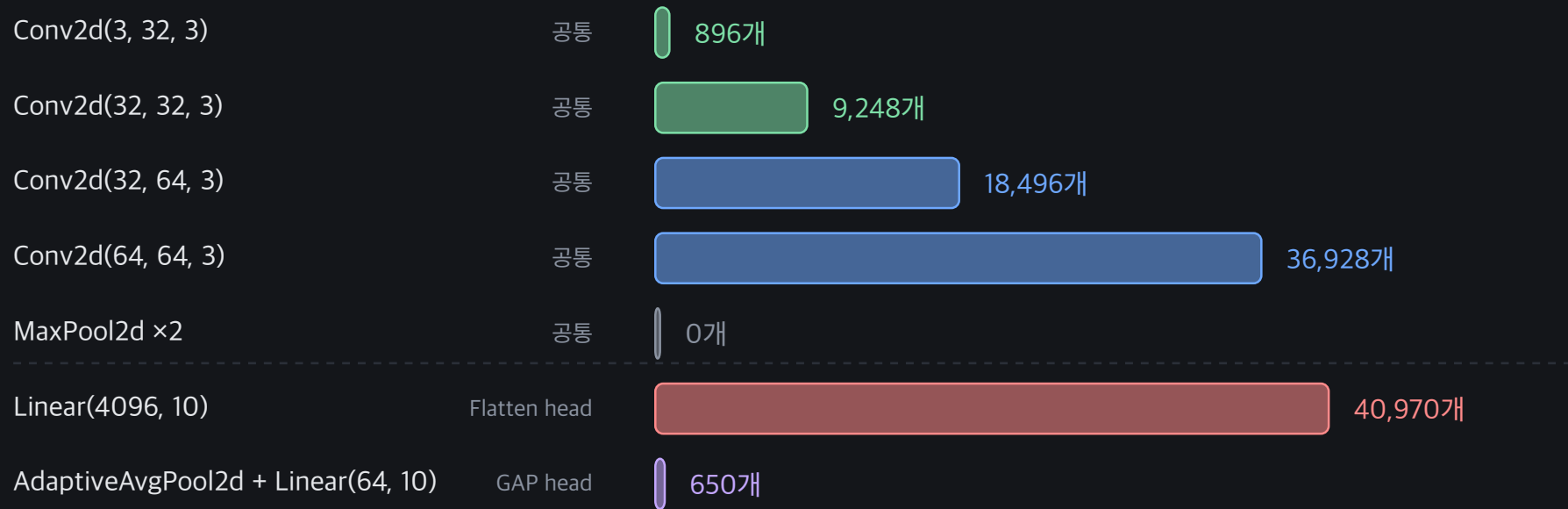
layer	type	output	params
block1.0	Conv2d	1x32x32x32	896
block1.2	Conv2d	1x32x32x32	9,248
block1.4	MaxPool2d	1x32x16x16	0
block2.0	Conv2d	1x64x16x16	18,496
block2.2	Conv2d	1x64x16x16	36,928
block2.4	MaxPool2d	1x64x8x8	0
gap	AdaptiveAvgPool2d	1x64x1x1	0
flatten	Flatten	1x64	0
classifier	Linear	1x10	650
전체 파라미터 66,218			

“ ReLU 줄은 지면상 뺐음 — 전부 0개

”

같은 합성곱, 다른 분류기

합성곱 네 층은 두 모델이 공유함 · 막대 길이가 곧 파라미터 수



Flatten head 전체	106,538	그중 마지막 Linear	40,970	(38.5%)
GAP head 전체	66,218	그중 마지막 Linear	650	(1.0%)

Conv는 면적이 파라미터에 안 들어가고, Linear는 입력 길이가 그대로 들어감

파라미터 수의 차이는 확인된 사실임. 어느 쪽이 더 잘 맞는지는 여기서 알 수 없음

이 분포가 뜻하는 것

- 합성곱 네 층은 두 모델이 **공유함** — 65,568개
- 갈리는 건 뒤쪽 한 줄
 - Linear(4096, 10) **40,970개** — Flatten head 전체의 38.5%
 - Linear(64, 10) **650개** — GAP head 전체의 1.0%

“ 파라미터 수 차이는 확인된 사실임. 어느 쪽이 더 잘 맞히는지는 여기서 알 수 없음 ”

summary 가 말하지 않는 것

- 계산 시간이 어디에 몰리는지
- 이 모델이 잘 맞힐지
- 층이 무엇을 배웠는지

“ shape과 파라미터 수를 확인하는 도구임. 성능 판단 도구가 아님

”

오늘 정리

그림의 한 칸	코드의 한 줄
N	끝까지 안 바뀜
Flatten	자리를 말할 수 없게 됨
손으로 적은 shape	입력 크기에 끌려감
파라미터	뒤쪽 한 줄에서 갈림

다음 질문

모델	만들었음
shape	맞음
파라미터	66,218개 / 106,538개

그런데 이 구조는

실제로 무엇을 배우는가?

학습시키고, 읽어 보기

CNN · 블록 11

지금까지 만든 것

블록	손에 들어온 것
6 · 7 · 8	합성곱 한 층
9	풀링과 GAP
10	층을 이어 붙인 모델 하나

아직 **한 번도 학습시키지 않았음**

이 강의에서
실제로 학습을 돌리는
유일한 시간임

오늘의 질문

이 구조는 실제로

무엇을 배웠는가?

오늘 돌릴 조건

학습 4,000장
검증 1,000장
테스트 1,000장 ← 마지막에 한 번만
배치 64 · epoch 5 · CPU

전체를 다 쓰지 않음. **끝나는 것이 먼저임**

먼저 확인할 것

실행하면 **데이터 출처**가 한 줄 찍힘

- 로컬 캐시가 있으면 그 데이터
- 없으면 **모양만 같은 대체 데이터**

|“ 나오는 숫자가 무엇에서 나온 값인지 항상 먼저 볼 것

”

돌리기 전에 정해 둘 것

무엇을 보면 **잘 돌아가는 것**인가

- 손실이 내려가는가
- 정확도가 0.1 근처를 벗어나는가

0.1은 10개 중 하나를 찍었을 때의 값임

돌리기 전에 눈금을 먼저 정한다

결과를 보고 나서 기준을 정하면 결과에 끌려간다



돌아가는 신호

- 손실이 내려간다
- 정확도가 0.1 을 확실히 넘는다
- train 과 val 이 같이 오른다

멈춰야 하는 신호

- 손실이 안 내려간다
- 정확도가 0.1 근처를 맴돈다
- val 만 내려가기 시작한다

숫자를 보고 기준을 정하면, 나온 숫자가 곧 기준이 된다

그래서 돌리기 전에 적어 둔다

실행이 끝났음

학습 정확도가 높으면

좋은 모델인가?

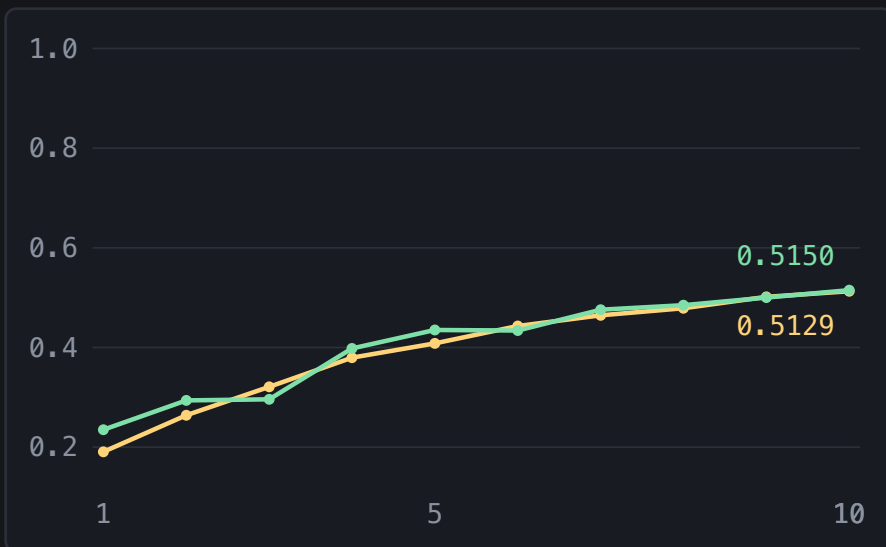
두 선이 벌어지는가, 붙어 있는가

같은 학습 데이터 · 같은 epoch. 분류기에 파라미터가 몰린 쪽만 벌어짐

GAP 분류기 — 파라미터 66,218

마지막에 평균 한 번, Linear(64 → 10)

가로축 = epoch



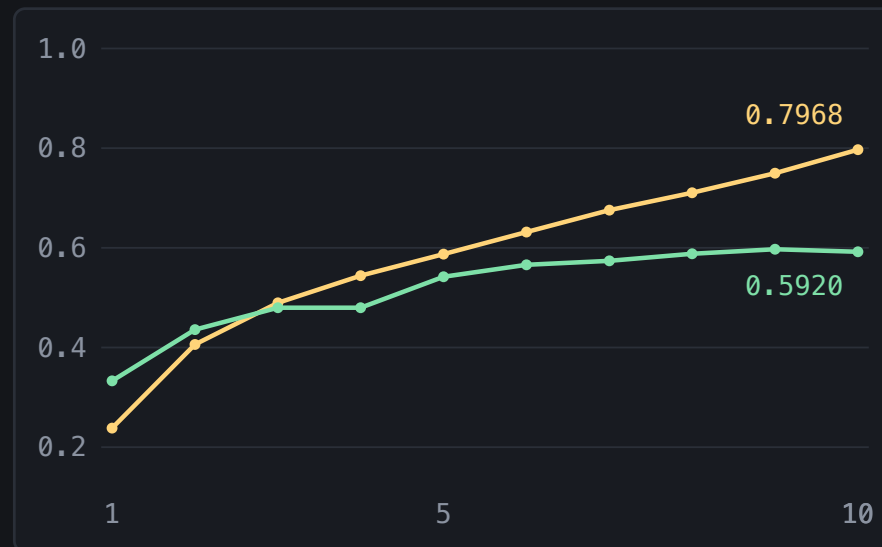
두 선이 붙어 있음

검증 쪽이 오히려 조금 높게 끝남 (0.5129 / 0.5150)

Flatten 분류기 — 파라미터 1,116,970

4096 → 256 → 10 을 통과

가로축 = epoch



학습만 계속 오르고 검증은 끝에서 꺾임

간격 $0.7968 - 0.5920 = 0.2048$

CIFAR-10 · 학습 10,000장 · 검증 1,000장 · epoch 10 · seed 2025 로 한 번 실행한 결과임
다른 데이터나 다른 seed에서는 숫자가 달라짐. 여기서 볼 것은 숫자가 아니라 두 선의 모양임

두 선을 읽는 법

모양	뜻
둘 다 올라감	아직 배우는 중
학습만 올라감	학습 데이터 쪽으로만 맞춰지는 중
둘 다 멈춤	이 조건에서는 여기까지

간격 그 자체가

다음 블록의 이유임

왜 오른쪽만 벌어졌나

GAP 분류기	66,218 개
Flatten 분류기	1,116,970 개

같은 합성곱 부분, 다른 분류기

파라미터가 몰린 자리에서 간격이 생김

다음 질문

이 모델 안에서는

무슨 일이 벌어지고 있나?

feature map은 답이 아니라 조사 도구

첫 블록이 내놓은 (32, 16, 16) 중 네 장 · 각 맵의 최댓값을 9로 두고 눈금을 맞춤 · 대체 데이터로 실행한 기록

입력 (요약)

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	1	0	1	0
0	0	0	0	4	6	5	0
0	0	1	3	5	5	5	2
1	4	5	5	5	6	7	2
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

발목 부츠 한 장

3번 맵

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	2	1	2	1
0	0	0	2	6	7	7	2
0	2	3	6	6	7	7	2
2	6	6	7	7	9	9	3
0	0	1	1	0	1	1	0
0	0	0	0	0	0	0	0

물체 자리에서만 큼

10번 맵

5	5	5	5	5	5	5	5
5	5	5	5	5	5	5	5
5	5	5	6	6	9	7	6
5	5	6	8	4	2	1	8
7	6	6	3	3	4	4	7
8	5	4	3	6	8	5	7
6	8	8	8	9	9	9	6
5	5	5	5	5	5	5	5

빈 곳에서도 값이 있음

26번 맵

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

이 장에서는 전부 0

여기서 말할 수 있는 것

- 이 입력에 대해 어떤 자리에서 값이 컸는지
- 32장 중 6장은 이 입력에서 한 번도 켜지지 않았다는 것

여기서 말할 수 없는 것 — 보기 좋은 맵이 중요한 맵이라는 것

세 장이 서로 다름

- 물체 자리에서만 큰 맵
- 빈 곳에서도 값이 있는 맵
- 이 장에서는 **한 칸도 커지지 않은 맵**

32장 중 6장이 마지막에 해당함

여기서 멈출 것

feature map은 내부 반응을 들여다보는 도구임

- 어떤 입력에서 어디가 컷는지는 알 수 있음
- 그 맵이 중요한지는 알 수 없음

“ 보기 좋은 그림이 곧 중요한 특징이라는 결론으로 가지 않음 ”

다음 질문

틀린 것들은

아무렇게나 틀렸을까?

틀린 483장이 하는 말

테스트 1,000장 중 틀린 것만 모아 정답과 예측을 짝지음 · CIFAR-10 · 학습 10,000장 · epoch 10

많이 나온 혼동 짝

cat	→	dog	31
airplane	→	ship	22
dog	→	cat	20
cat	→	frog	20
ship	→	airplane	19
truck	→	automobile	18

확신까지 붙은 오답

정답	예측	예측 확률	정답 확률
airplane	ship	0.897	0.096
deer	horse	0.878	0.077
truck	automobile	0.877	0.059
airplane	ship	0.865	0.084
dog	frog	0.852	0.016

틀리는 자리가 고르지 않음 — 겉모습이 비슷한 짝에 몰려 있음

확률 0.9 에 가까운 오답도 있음. 확률은 모델의 확신이지 정답률이 아님

정확도 한 숫자만 보면 이 구조가 보이지 않음

틀림에도 결이 있음

- 겉모습이 비슷한 짝에 몰려 있음
- 확률 1.000짜리 오답도 있음

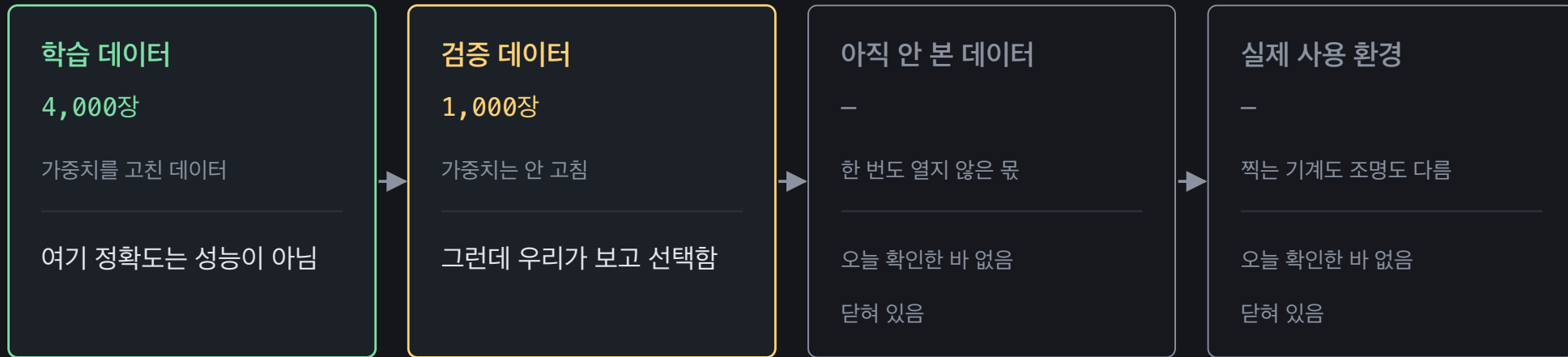
확률은 모델의 **확신**이지 정답률이 아님

그래서

이 모델은

몇 점짜리 모델인가?

지금 확인한 것은 어디까지인가



오늘의 숫자는 검증 데이터에서 나온 값임

검증 데이터를 보면서 epoch 수와 구조를 고른 순간, 그 데이터도 선택에 쓰인 것임

그래서 마지막 판단은 한 번도 열지 않은 문으로 해야 함

실전 성능은 그것과도 또 다름. 오늘 다루지 않음

검증 데이터의 이상한 위치

가중치를 고치는 데는 쓰지 않았음

그런데 **우리가 그 숫자를 보고 골랐음**

- epoch을 몇 번 돌릴지
- 어떤 구조를 쓸지

| “ 보는 순간 그 데이터도 선택에 쓰인 것임

”

오늘 말할 수 있는 것

말할 수 있음	이 조건에서 두 선이 이렇게 움직였다
말할 수 없음	이 모델의 실제 성능
말할 수 없음	다른 데이터에서도 이럴 것이다

오늘 정리

손실 곡선	두 선의 간격 을 봄
feature map	반응 조사 도구
오답	비슷한 짝에 몰림
검증 점수	최종 판정이 아님

남은 문제

학습 정확도만 계속 올라가는 그림을 봤음

이 간격을

줄일 방법이 있는가?

다음 시간

한 가지 이상한 방법을 봄

학습할 때는 모델의 일부를 꺼 두고,
평가할 때는 전부 켜는 방법

학습 때와 평가 때

모델이 달라져도 되는가?

일부를 꺼 두고 배우기

CNN · 블록 12

앞 시간에 본 그림

학습 정확도	0.7968
검증 정확도	0.5920

학습 쪽만 계속 올라갔음

이 **간격**이 오늘의 출발점임

오늘의 질문

학습 때와 평가 때

모델이 달라져도 되는가?

오늘의 아이디어 한 줄

학습할 때는

일부러 일부를 끈다

왜 그게 도움이 되는지는 잠시 뒤에

매번 다른 일부를 끈다

같은 층, 같은 가중치. 달라지는 것은 이번 스텝에 누가 쉬는가

1번째 스텝



켜진 4개

2번째 스텝



켜진 4개

3번째 스텝



켜진 4개

갱신

갱신

꺼졌다는 것은

이번 계산에서 그 유닛의 출력이 0
그래서 이번 갱신에도 참여하지 않음

꺼졌다는 것이 아닌 것

연결이 잘려 나간 것이 아님
다음 스텝에 다시 켜질 수 있음

가중치는 그대로 남아 있음. 이번 스텝에 쓰이지 않았을 뿐임

그래서 “가지치기”나 “삭제”와는 다른 이야기임

매 스텝마다 다시 뽑음

- 이번에 쉰 유닛이 다음에도 쉬는 것이 아님
- 확률 p 로 각자 독립으로 정해짐

같은 층을 계속 다른 모양으로 쓰는 셈임

꺼진 유닛은

잘려 나간 연결이 아님

가중치는 그대로 남아 있음

그런데

끄기만 하면

다음 층이 받는 총량은?

손으로 세어 보기

원래 값	2	4	6	8	합	20
절반이 꺼짐	2	0	6	0	합	---

절반을 끄면 총량은 대략 몇 배가 되는가?

끄기만 하면 총량이 줄어든다

$p = 0.5$ · 값 네 개짜리 총 하나로 직접 확인한 기록

원래 값

2

4

6

8

합 20

여섯 번 뽑아 본 결과

살아남은 값에

$1/(1-p) = 2$ 를 곱함

4

8

0

0

합 12

0

0

12

0

합 12

0

0

12

16

합 28

4

8

12

0

합 24

4

0

12

0

합 16

0

0

0

16

합 16

여섯 번의 합

$12 \cdot 12 \cdot 28 \cdot 24 \cdot 16 \cdot 16$

평균 18.0

원래 합 20 근처에서 흔들림

여섯 번만 뽑은 결과임

많이 뽑을수록 가까워짐

끄면서 동시에 키우기 때문에, 다음 총이 받는 총량이 크게 흔들리지 않음

맞춰 주는 것은 평균 수준임. 매번의 값은 여전히 달라짐

남은 값을 키움

$$1 / (1 - p)$$

$p = 0.5$ 면 살아남은 값이 **2배**가 됨

끄면서 동시에 키우기 때문에 총량이 크게 흔들리지 않음

확률을 바꿔 가며 확인

p=0.1	실제 0 비율 0.1004	살아남은 값 1.1111
p=0.3	실제 0 비율 0.3000	살아남은 값 1.4286
p=0.5	실제 0 비율 0.5006	살아남은 값 2.0000
p=0.8	실제 0 비율 0.8005	살아남은 값 5.0000

배율이 $1/(1-p)$ 와 그대로 맞음

다음 질문

평가할 때도

일부를 꺼야 하는가?

학습 때와 평가 때 다르게 동작한다

같은 층 · 같은 입력 · 바뀐 것은 모드 하나

`model.train()`

일부를 끄고, 남은 값을 키움



같은 입력을 두 번 넣으면

결과가 서로 다름

p 별로 실제 확인한 배율

p=0.1	p=0.3	p=0.5	p=0.8
×1.1111	×1.4286	×2.0000	×5.0000

`model.eval()`

전부 켜고, 아무것도 하지 않음



같은 입력을 두 번 넣으면

결과가 완전히 같음

입력과 출력이 같은지 직접 확인함

`torch.equal(x, out) → True`

학습 때 미리 키워 두었기 때문에 여기서 줄일 것이 없음

평균 입력 0.0013 · 학습 모드 0.0012 · 평가 모드 0.0013

표준편차 입력 1.0004 · 학습 모드 1.4143 · 평가 모드 1.0004

맞춰지는 것은 평균 쪽임. 흩어짐은 학습 중에 오히려 커짐

평가 모드에서는 아무 일도 안 함

입력이 그대로 나옴

```
torch.equal(x, out) → True
```

학습할 때 미리 키워 뒀기 때문에
평가할 때 따로 줄일 것이 없음

평가가 매번 달라지면 곤란함

- 같은 사진을 두 번 넣었는데 답이 다름
- 어제 낸 점수와 오늘 낸 점수가 다름

평가는 **한 번 정해지면 같아야 함**

코드에서는 어디에 들어가나

```
nn.Flatten(),  
nn.Dropout(p=0.5),  
nn.Linear(64 * 8 * 8, 256), nn.ReLU(),  
nn.Dropout(p=0.3),  
nn.Linear(256, 10),
```

앞 시간에 **간격이 생겼던 바로 그 자리임**

층별 표에서 보이는 것

Dropout 입력 4096 → 출력 4096 params 0

- 모양을 바꾸지 않음
- 학습할 것이 하나도 없음

“ 배우는 층이 아니라 학습 중에만 켜지는 장치임

”

회수 질문

블록 4에서 이 둘을 나눠 줬음

`model.eval()` 과

`torch.no_grad()` 는

같은 스위치인가?

서로 다른 두 개의 스위치

블록 4에서 나눠 둔 구분을 여기서 다시 씀

기울기를 쌓는가

`torch.no_grad()`

켜면 — 되돌아갈 길을 기록함

끄면 — 기록하지 않음

메모리와 시간의 문제임

Dropout이 켜지고 꺼지는 것과

아무 상관이 없음

층이 어떻게 동작하는가

`model.train()` / `model.eval()`

train — Dropout이 일부를 끄

eval — Dropout이 그대로 통과시킴

출력값 자체가 달라짐

기울기를 켜다고 해서

여기가 바뀌지는 않음

직접 확인한 결과

`model.train()` 만

출력이 매번 달라짐

기울기 기록됨

`model.train() + no_grad()`

출력이 매번 달라짐

기울기 기록 안 됨

`model.eval()` 만

출력이 항상 같음

기울기 기록됨

평가할 때는 둘 다 필요함 — `eval()`로 층을 바꾸고, `no_grad()`로 기록을 끄

서로 다른 두 가지를 끄

	무엇을 바꾸나
<code>train()</code> / <code>eval()</code>	층의 동작
<code>no_grad()</code>	기울기 기록

평가할 때는 둘 다 필요함

자주 나는 사고

`model.eval()` 을 부르지 않고 평가함
→ 평가 중에도 일부가 꺼짐
→ 점수가 실제보다 낮게, 그리고 매번 다르게 나옴

에러가 나지 않음. **조용히 틀림**

실제로 넣어 본 결과

	학습 정확도	검증 정확도	검증 손실	간격
없음	0.9453	0.5100	2.6905	0.435
Dropout 넣음	0.6823	0.5290	1.3573	0.153

학습 쪽이 크게 **내려갔음**. 간격이 0.435 → 0.153

검증 손실은 **절반**, 검증 정확도는 **올라감**

Dropout 은 외우는 걸 막는다

CIFAR-10 · 학습 4,000 · epoch 15 · 같은 구조에 Dropout 만 추가

Dropout 없음

간격 0.435

train 0.95

val 0.51

Dropout 있음

간격 0.153

train 0.68

val 0.53

간격 0.435 → 0.153 · 검증 손실 2.69 → 1.36 · 검증 정확도 0.510 → 0.529

학습 정확도를 26%p 포기하고 검증을 얻었다

단일 실행 한 번의 기록. seed 를 바꾸면 숫자는 달라진다

이 표가 말하지 않는 것

- 한 번의 실행, 한 개의 seed
- 하나의 데이터, 하나의 구조

“ 이 조건에서 이랬다는 기록임
Dropout이 항상 이득이라는 뜻이 아님

”

오늘 정리

학습 때	일부를 끄고, 남은 값을 $1/(1-p)$ 배
평가 때	전부 켜고 그대로 통과
파라미터	없음
넣는 자리	파라미터가 몰린 곳

다음 시간

우리가 만든 모델은 작은 사진 10종짜리임

이 골격을 아주 크게 키우면 어떻게 되는가

규모가 커질 때

무엇이 새로 필요해졌는가?

1998에서 2012로

CNN · 블록 13 · LeNet과 AlexNet

우리가 지금 들고 있는 것

conv → conv → pool → conv → conv → pool → 분류기

- 작은 사진 10종
- 파라미터 백만 개 남짓
- 노트북 CPU에서 몇 분

오늘의 질문

규모가 커질 때

무엇이 새로 필요해졌는가?

1998과 2012 사이에 놓인 14년

같은 구조를 크게 키우는 데 왜 그만큼 걸렸는가

LeNet-5

손글씨 숫자 · 32×32 흑백
conv → subsample → 분류

구조의 아이디어는 이어짐

AlexNet

사진 1,000종 · 224×224 컬러
같은 골격을 크게 키움

1998

2012

구조를 몰라서 못 한 것이 아님

커진 문제를 감당하려면 무엇이 새로 있어야 했는가 — 오늘의 질문

14년

구조를 몰라서 걸린 시간이 아님

오늘의 근거

LeNet-5

“ LeCun, Bottou, Bengio, Haffner (1998)
Gradient-Based Learning Applied to Document Recognition
Proceedings of the IEEE 86(11) ”

여기 나오는 총 이름과 숫자는 전부 이 논문에 적힌 값임

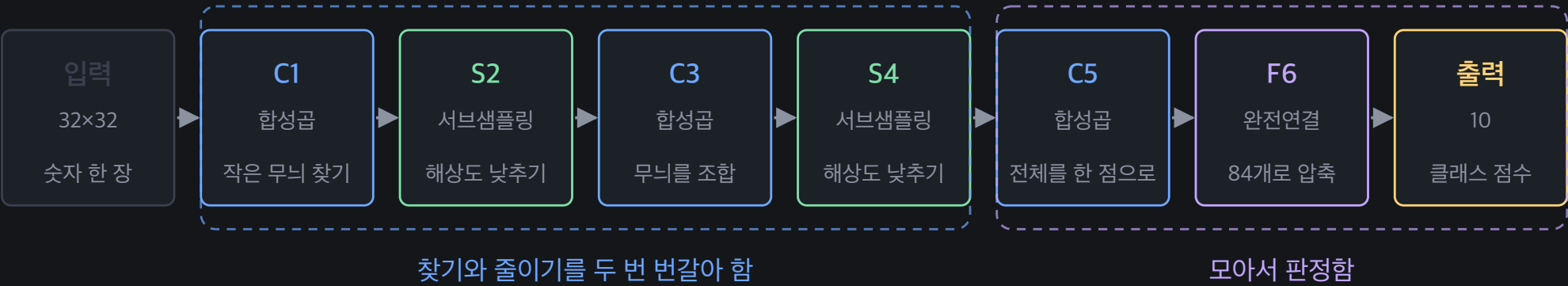
첫 질문

1998년의 그물망은

어떤 순서였는가?

LeNet-5 — 총 일곱 개의 순서

LeCun 외 1998, Proceedings of the IEEE 86(11), Fig. 2



오늘 우리가 쓰는 골격과 순서가 같음 — 특징을 찾고, 줄이고, 마지막에 분류함

이름과 세부 연산은 다름. 같은 것은 순서임

오늘 우리가 쓰는 순서와 같음

찾는다 → 줄인다 → 찾는다 → 줄인다 → 판정한다

- 논문 표기로는 C1 · S2 · C3 · S4 · C5 · F6 · 출력
- 입력을 빼면 **일곱 층**

다음 질문

층을 지날 때

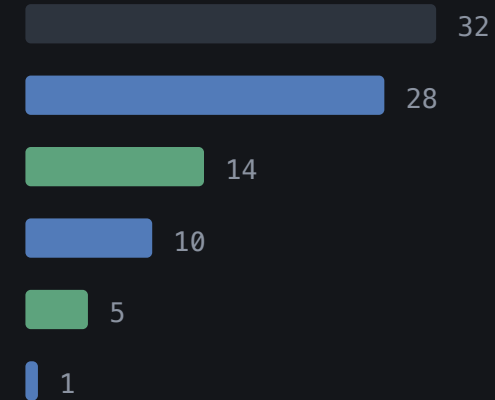
크기는 어떻게 변하는가?

채널은 늘고 면적은 줄어든다

LeCun 외 1998, Fig. 2 및 Section II-B에 적힌 값

층	맵 수	한 맵의 크기	만드는 연산
입력	1	32 × 32	—
C1	6	28 × 28	5×5 합성곱
S2	6	14 × 14	2×2 서브샘플링
C3	16	10 × 10	5×5 합성곱
S4	16	5 × 5	2×2 서브샘플링
C5	120	1 × 1	5×5 합성곱
F6	84	—	완전연결
출력	10	—	클래스 하나당 유닛 하나

한 번의 길이



면적은 32 → 1 로 줄고, 맵 수는 1 → 120 으로 늘어남

어디에 있는지를 버리는 대신, 무엇이 있는지를 더 많이 들고 감

두 방향이 반대임

	입력	끝
한 맵의 한 번	32	1
맵의 개수	1	120

어디에 있는지를 버리고 무엇이 있는지를 늘림

잠깐 계산

S2 의 맵 한 변은 14
5×5 창을 패딩 없이 한 칸씩 옮기면 C3 는?

블록 7에서 쓰던 그 계산임

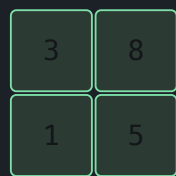
그런데

1998년의 "줄이기"는

우리가 쓰는 max pooling인가?

1998의 서브샘플링은 오늘의 max pooling이 아니다

S2 · S4 서브샘플링 (LeNet-5)



2×2 창 · 겹치지 않음

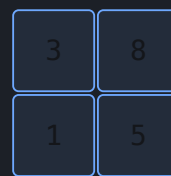
네 값을 더함
학습되는 계수를 곱함
학습되는 편향을 더함
시그모이드를 통과시킴

값 하나를 뽑는 것이 아니라

학습되는 파라미터를 가진 연산임

S2는 12개, S4는 32개의 학습 파라미터를 가짐

MaxPool2d (오늘)



2×2 창 · 겹치지 않음

가장 큰 값 하나를
그대로 내보냄
곱하는 계수 없음
더하는 편향 없음

학습 파라미터가 하나도 없음

고정된 규칙임

블록 9에서 파라미터 0을 직접 확인했음

둘 다 “작게 만든다”. 하지만 같은 연산이 아님

옛 그림에서 pooling이라는 말을 보면 무엇으로 줄였는지 확인할 것

결정적인 차이

	LeNet 서브샘플링	MaxPool2d
하는 일	더하고 곱하고 더함	최댓값 하나
학습 파라미터	있음	없음

논문은 S2에 12개, S4에 32개가 있다고 적음

이름이 비슷하다고
같은 연산이 아님

1998년까지 성립한 것

conv → subsample → classify

이 순서가 **하나의 틀**로 자리를 잡음

여기까지가 오늘의 앞쪽 절반임

뒤쪽 절반

그 사이 14년 동안

무엇이 바뀌었는가?

커진 문제가 만든 네 가지 요구

구조가 아니라 규모가 만든 문제들

문제

숫자 10종에서
사물 1,000종으로

필요

훨씬 많은
라벨 데이터

01

문제

32×32 흑백에서
224×224 컬러로

필요

감당 가능한
연산 장치

02

문제

층이 깊어지자
학습이 느려짐

필요

포화하지 않는
활성 함수

03

문제

파라미터가 많아
학습에만 맞춰짐

필요

과하게 맞추는 것을
막는 장치

04

AlexNet은 ReLU를 만든 것도, 처음 쓴 것도 아님

논문은 Nair·Hinton의 이름을 따라 ReLU라고 부른다고 직접 밝히고,
자기들이 최초가 아니라고도 적어 둬. 큰 CNN에서 이 선택을 굳힌 쪽임

네 가지가 한꺼번에 필요했음

커진 것	필요해진 것
데이터	백만 장 규모의 라벨
계산	감당 가능한 장치
깊이	포화하지 않는 활성 함수
파라미터	과하게 맞추는 것을 막는 장치

자주 도는 잘못된 문장

| “ AlexNet이 ReLU를 만들었다 ”

사실이 아님

논문이 직접 적어 둔 것

- Nair와 Hinton을 **따라서** ReLU라고 부른다고 밝힘
- 신경망의 활성화 함수를 바꾼 것이 자기들이 처음이 아니라고 적음

큰 CNN에서 이 선택을 굳힌 쪽이라고 말하는 것이 정확함

앞 시간과 이어지는 자리

AlexNet 논문은 **완전연결 앞 두 층에 dropout을 넣었다고** 밝힘

- 파라미터가 몰린 자리
- 우리가 어제 넣었던 바로 그 자리

“ 같은 문제에 같은 자리의 해법임 ”

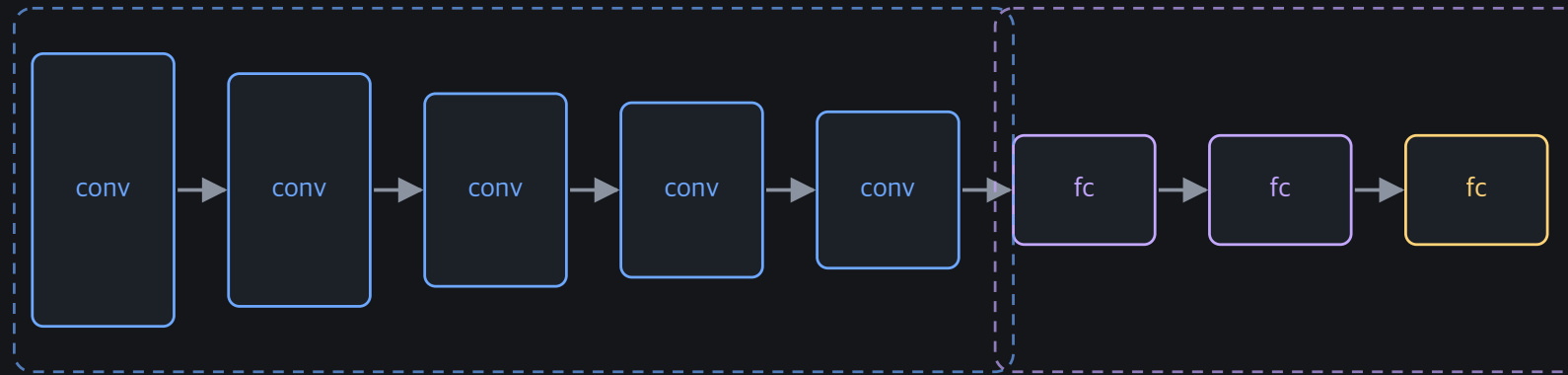
그래서

AlexNet은

어떻게 생겼는가?

AlexNet — 실루엣만 보기

Krizhevsky 외 2012. 층별 수치를 외우는 것이 목적이 아님



합성곱 다섯 층 — 특징을 찾는 부분

완전연결 세 층 — 판정하는 부분

앞의 두 층에 dropout을 넣었다고 논문이 밝힘

LeNet-5와 같은 골격 — 찾는 부분과 판정하는 부분

달라진 것은 층 수와 폭, 그리고 커진 규모를 감당하려고 붙인 장치들

여기서 층별 숫자를 외울 필요는 없음

골격은 그대로임

특징을 찾는 부분 → 판정하는 부분

달라진 것은 **총 수와 폭**, 그리고 규모를 감당하려고 붙인 장치들

오늘 숫자를 외우지 않는 이유

총별 채널 수를 외워도 다음 구조에 쓸 데가 없음

남길 것은 이것 하나임

문제가 있었고

그에 맞는 도구가 붙었다

대회 성적을 인용하지 않는 이유

같은 "정확도"라도 조건이 다르면 다른 숫자임

- 어떤 과제인지
- 여러 모델을 합쳤는지
- 한 장을 몇 번 잘라 넣었는지

“ 조건 없이 숫자만 옮기면 비교가 되지 않음

”

오늘 다루지 않은 것

LeNet 구현

GPU 두 장에 나눠 엮는 방법

정규화 층의 세부

전체 코드

연도별 목록

오늘 정리

1998	찾기 · 줄이기 · 판정하기의 틀
서브샘플링	max pooling과 다른 연산
2012	같은 골격 · 커진 규모
ReLU	발명이 아니라 정착

다음 시간

AlexNet의 첫 합성곱은 11×11 짜리 큰 창이었음

그다음 세대는 3×3 만 반복하기 시작함

작은 3×3 만 쌓아도

넓게 볼 수 있는가?

반복되는 3×3 , 그리고 1×1

CNN · 블록 14

지금 어디까지 왔는가

블록 6~8	합성곱 한 번이 무엇을 하는지
블록 9~11	부품을 모아 모델을 만들고 학습
블록 13	규모가 커지자 새로 필요해진 것들
오늘	커널 크기를 매번 골라야 하는가

오늘은 새 연산을 배우지 않음

이미 아는 합성곱을 **어떻게 배치할지**의 문제임

오늘의 질문

작은 3×3 만 쌓아도

넓게 볼 수 있는가?

넓게 보고 싶을 때 떠오르는 방법

큰 커널을 쓰는 것

11 × 11

5 × 5

7 × 7

한 번에 넓은 영역을 본다

정말 이 방법밖에 없는가?

먼저 셀 것을 정함

출력 피처맵의 **값 하나**를 고름

그 값이 만들어지기까지 **원본에서 몇 칸을 근거로 삼았는가**

| “ 커널 크기와 다름. 커널은 한 층의 창, 이건 **누적된 범위임**

”

3×3만 쌓아도 보는 범위는 넓어진다

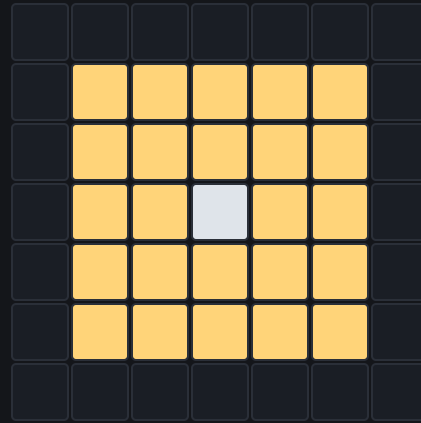
한 출력값이 원본에서 몇 칸을 근거로 삼는가



3×3 한 층

근거가 되는 원본 영역 3×3

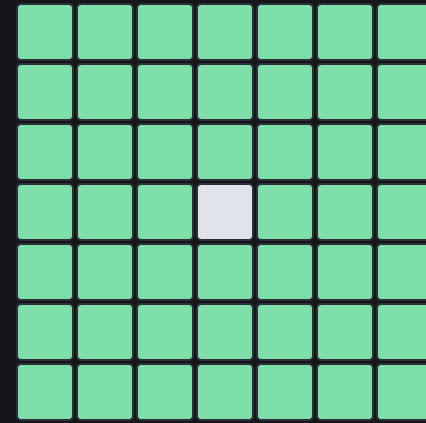
+1층
→



3×3 두 층

근거가 되는 원본 영역 5×5

+1층
→



3×3 세 층

근거가 되는 원본 영역 7×7

흰 칸 = 출력값 하나 · 색칠한 칸 = 그 값이 근거로 삼은 원본 자리

층을 하나 쌓을 때마다 한 번이 2칸씩 늘어남 → 3 · 5 · 7 · 9 ...

손으로 세어 보기

3×3	한 층	→	3
3×3	두 층	→	?
3×3	세 층	→	?
3×3	네 층	→	?

층 하나를 얹으면 한 변이 몇 칸 늘어나는가

답

1층	3	2층	5	3층	7	4층	9
----	---	----	---	----	---	----	---

한 층마다 2칸씩

3×3 세 층의 범위 = 7×7 한 층의 범위

“ stride가 1일 때의 이야기임. stride가 커지면 더 빨리 넓어짐 ”

그렇다면

같은 7×7 을 보는데

어느 쪽이 더 싼가?

먼저 예상해 보기

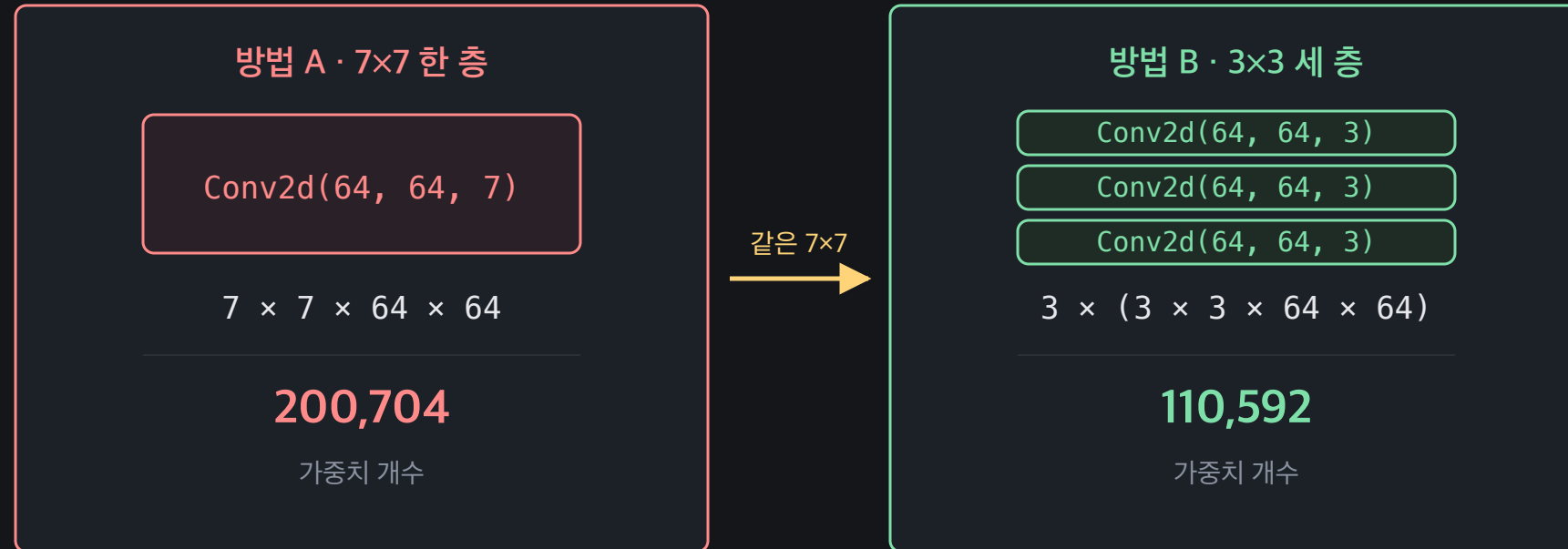
입력 채널 64, 출력 채널 64로 고정

- A. 7×7 한 층
- B. 3×3 세 층

가중치 개수가 더 적은 쪽은?

같은 7×7을 보는 두 가지 방법

채널 64 → 64 기준. 가중치만 센 값 (bias 제외)



$49C^2$ 대 $27C^2$ → 어떤 채널 수를 넣어도 비율은 $27/49 \approx 0.55$

같은 범위를 볼 때만 참인 비교임. 층이 늘어나면 총량은 얼마든지 커질 수 있음

일반화

7×7 한 층	$49 \times C \times C$
3×3 세 층	$27 \times C \times C$

채널 수와 무관하게 비율은 $27 / 49 \approx 0.55$

층을 셋으로 나눴는데 가중치는 오히려 **줄어듦**

이 비교의 조건

같은 범위를 만들 때만 성립함

- 같은 채널 수
- 같은 최종 범위 7×7

이 조건을 빼면 결론도 사라짐

“ 뒤에서 이 조건을 빼고 말했을 때 무슨 일이 생기는지 봄 ”

다음 질문

이 사실을 알고 나면

설계가 어떻게 바뀌는가?

VGG가 정한 규칙은 네 줄이다

새로운 연산이 아니라, 이미 있는 연산을 쓰는 문법

① kernel은 3×3 하나로 고정

크기를 매번 고르지 않음

② padding=1 로 크기 유지

블록 안에서는 공간이 안 줄어듦

③ 블록 끝에서만 pool

여기서만 한 번이 절반

④ 공간이 절반이면 채널은 2배

줄어든 자리 수를 채널로 갯음

블록 하나의 모양

3×3 conv

ReLU

3×3 conv

ReLU

MaxPool 2

이 묶음을 5번 반복하면 VGG16

커널 크기를 고민하는 문제를 “몇 층 쌓을까” 하나로 바꾼 것

문법 네 줄

1. 커널은 **3×3 하나로 고정**
2. `padding=1` 로 블록 안에서는 크기 유지
3. 블록 **끝에서만** pool
4. 공간이 절반이면 **채널은 2배**

암기할 아키텍처가 아니라 **반복 가능한 규칙**임

블록 하나를 코드로 보면

```
nn.Sequential(  
    nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),  
    nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2),  
)
```

- 첫 줄에서만 채널이 바뀜
- padding=1 이 크기를 지킴
- MaxPool2d(2) 가 한 번을 절반으로

이 묶음을 다섯 번

3 → 64	conv 2개
64 → 128	conv 2개
128 → 256	conv 3개
256 → 512	conv 3개
512 → 512	conv 3개

conv 13개 + Linear 3개 = **가중치를 가진 총 16개**

코드를 읽을 때 주의

돌아다니는 구현에는 `BatchNorm2d` 가 끼어 있는 경우가 많음

```
nn.Conv2d(...), nn.BatchNorm2d(...), nn.ReLU()
```

- 원래 발표된 구성에는 이 줄이 없음
- 학습을 안정시키는 장치임. **오늘은 다루지 않음**
- 이런 코드는 **VGG 스타일 · BN 변형**으로 부르기

다음 질문

블록을 지날 때마다

무엇이 변하는가?

블록을 지날 때마다 무엇이 변하는가

입력 224×224 컬러 한 장 · torch 실측

	그 블록을 나온 shape	conv 층	파라미터
입력	3 × 224 × 224	—	—
블록 1	64 × 112 × 112	3×3 ×2	38,720
블록 2	128 × 56 × 56	3×3 ×2	221,440
블록 3	256 × 28 × 28	3×3 ×3	1,475,328
블록 4	512 × 14 × 14	3×3 ×3	5,899,776
블록 5	512 × 7 × 7	3×3 ×3	7,079,424

공간은 절반씩 줄고, 채널은 2배씩 늘어남

파라미터는 뒤쪽 블록에 몰려 있음 — 채널이 커졌기 때문임

표에서 읽을 것 두 가지

공간	224 → 112 → 56 → 28 → 14 → 7
채널	3 → 64 → 128 → 256 → 512 → 512

- 자리 수는 줄고, 자리마다 든 값은 늘어남
- **파라미터는 뒤쪽 블록에 몰려 있음**

“ 자리가 줄었다고 계산량이 같은 비율로 주는 것은 아님 ”

그런데 자주 보는 문장이 있음

"VGG는 AlexNet보다

파라미터가 적다"

맞는 말인가?

답하기 전에

지금까지 본 것

- 3×3 반복은 큰 커널보다 가중치가 적다
- VGG는 3×3 만 쓴다

그래서 자연스럽게 나오는 결론이 저 문장임

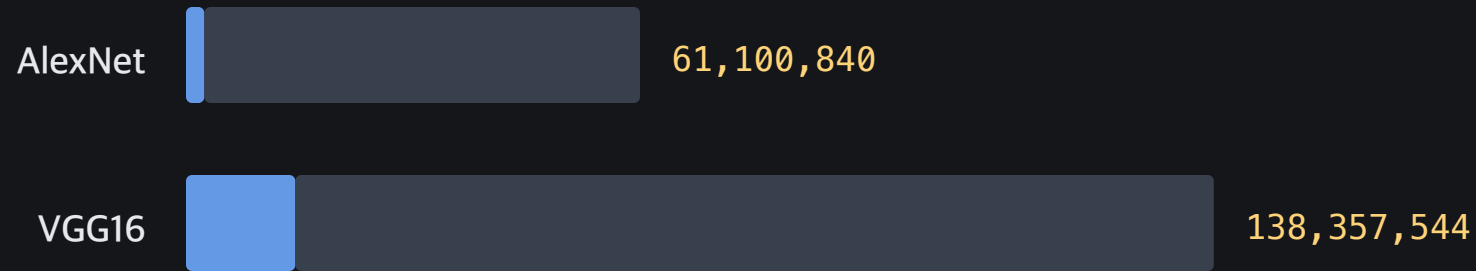
직접 세어 보면 어떻게 되는가

“VGG는 AlexNet보다 파라미터가 적다”

16층 기준으로 직접 세어 보면 어떻게 되는가

합성곱

Linear



합성곱 부분만	AlexNet 2,469,696	VGG16 14,714,688
뒤쪽 Linear 부분	AlexNet 58,631,144	VGG16 123,642,856

16층 구성에서 VGG16 전체는 AlexNet의 약 2.3배. 적지 않음
반복 3×3이 줄이는 것은 같은 범위를 큰 커널로 만든 경우와 비교했을 때뿐임
두 모델은 층 수도 채널 수도 다름 — 애초에 같은 조건의 비교가 아님

세어 본 결과

AlexNet	61,100,840
VGG16	138,357,544

일반적인 16층 구성에서 VGG16이 **약 2.3배 많음**

앞의 문장은 이 구성에서 **거짓임**

그러면 무엇이 참인가

주장	참인가
반복 3×3이 항상 파라미터를 줄인다	아님
같은 범위를 큰 커널로 만든 경우보다 적다	맞음

조건을 떼어내면 참이 거짓이 됨

“ 두 모델은 층 수도 채널 수도 다름. 애초에 통제된 비교가 아니었음 ”

다음 질문

자리는 그대로 두고

채널만 바꾸려면?

이런 상황이 생김

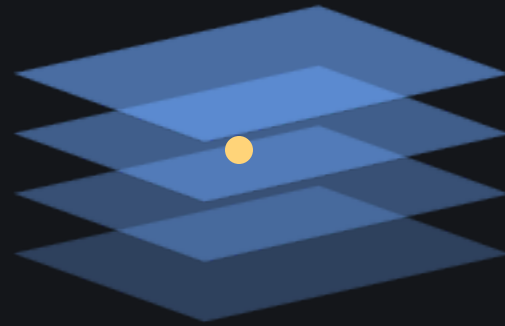
지금 손에 있는 것	$256 \times 8 \times 8$
필요한 것	$64 \times 8 \times 8$

- 자리 수는 그대로여야 함
- 자리마다 든 값 목록만 짧아지면 됨

3×3을 쓰면 이웃 자리까지 섞임. **그건 원하는 게 아님**

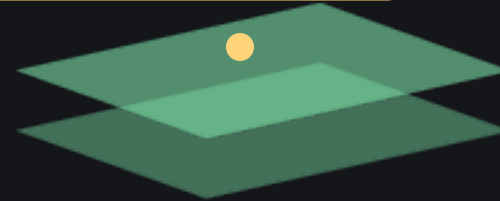
1x1 Convolution은 작은 합성곱이 아니다

자리는 그대로 두고, 그 자리의 값 목록만 바꾼다



입력
 $256 \times 8 \times 8$
채널 256겹

Conv2d(256, 64, kernel_size=1)



출력
 $64 \times 8 \times 8$
채널 64겹

한 자리 (h, w) 를 관통하는 값 256개를
섞어서 값 64개로 다시 씀
옆 자리와는 섞지 않음 · H·W는 손대지 않음

H·W 그대로 · 채널만 다시 씀 · 줄일 수도 늘릴 수도 있음

채널만 바꾸는 데 3×3을 쓸 이유는 없다

같은 256 → 64 변환. torch 실측

`Conv2d(256, 64, 3, padding=1)`

$3 \times 3 \times 256 \times 64$

147,456

`Conv2d(256, 64, 1)`

$1 \times 1 \times 256 \times 64$

16,384

이 도구를 기억해 둘 것

다음 블록에서
두 텐서를 더하려다가
shape가 안 맞는 일이
생김

1×1은 3×3의 정확히 1/9 — 커널 자리 수가 9분의 1이기 때문임

“채널 수를 원하는 값으로 바꾸되 자리는 건드리지 않기” — 이게 필요할 때 쓰는 도구

1×1을 넣는다고 성능이 좋아진다는 뜻은 아님. 무엇을 바꾸는 도구이지만 말하는 것임

오늘 정리

3×3 쌓기	한 층마다 범위가 2칸씩
3×3 세 층	7×7 한 층과 같은 범위 · 가중치 55%
VGG 문법	3×3 고정 · 블록 끝 pool · 채널 2배
파라미터	VGG16 > AlexNet · 조건 붙은 주장만 참
1×1	자리는 유지, 채널만 다시 씀

남는 문제

VGG는 깊게 쌓으면 좋아진다는 쪽으로 답했음

16층 → 19층 → 더?

계속 깊게 하면

계속 좋아지는가?

깊이의 문제

CNN · 블록 15

앞 블록에서 남긴 질문

VGG가 준 답

3×3만 쓰고, 대신 깊게 쌓는다

계속 깊게 하면

계속 좋아지는가?

오늘은 코드가 한 줄뿐임

나머지는 전부 그 한 줄이 왜 필요한가

오늘의 질문

더 깊게 했는데

학습조차 나빠지면

그건 무엇인가?

"결과가 나쁘다"에는 종류가 있음

같은 문장으로 부르면 다른 처방을 쓰게 됨

train 손실	학습에 쓴 데이터로 잰 값
dev 손실	학습에 쓰지 않은 데이터로 잰 값

두 값을 같이 봐야 이름을 붙일 수 있음

곡선 두 개를 같이 봐야 이름을 붙일 수 있다

정성적 모형 곡선 · 측정 결과가 아님



— train - - - dev

곡선 하나만 보면 셋을 구분할 수 없음

이름 붙이는 규칙

관찰	이름
train도 dev도 높은 채로 멈춤	underfitting
train은 낮는데 dev가 올라감	overfitting
둘 다 낮고 붙어 있음	잘 맞은 경우

곡선 하나만 보고는 셋을 구분할 수 없음

잠깐 연습

A.	train 0.05	dev 0.62	(dev는 중간부터 올라감)
B.	train 0.58	dev 0.60	

각각 무엇인가?

답

- | | |
|-----------------|--------------------|
| A. overfitting | 외운 것은 맞히고, 새것에서 틀림 |
| B. underfitting | 외운 것부터 못 맞힘 |

두 경우의 처방은 다름

A는 모델을 줄이거나 규제, B는 모델을 키우거나 더 학습

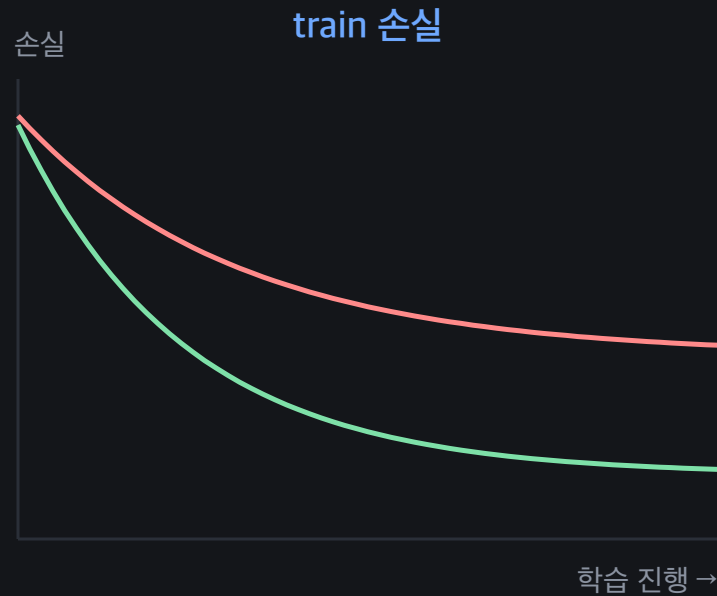
그런데

이건 셋 중

어느 것인가?

그런데 이건 셋 중 어느 것도 아니다

정성적 모형 곡선 · 측정 결과가 아님



학습에 쓴 데이터로 잴 값



안 쓴 데이터로 잴 값

— 얇은 모델 — 더 깊게 만든 모델

깊은 쪽이 **train 손실**부터 더 높음 — 외운 것조차 못 함
dev만 나쁜 것이 아니므로 overfitting이 아님. 이 상황을 degradation이라 부름

관찰된 것

같은 데이터 · 같은 학습 방식 · **층 수만 늘림**

overfitting	train은 좋는데 dev가 나쁨
degradation	더 깊게 했는데 train 자체가 더 나쁨

dev만 나쁜 게 아니므로 overfitting이 아님

이 곡선의 출처

- 화면의 곡선은 **미리 만들어 둔 모형 곡선**임
- 오늘 이 자리에서 측정한 값이 아님
- 자주 인용되는 "75% / 65%" 같은 숫자도 설명을 위한 **가정값**이지 측정 결과가 아님

| “ 모양을 보여 주는 그림임. 숫자를 외울 자리가 아님

”

왜 이상한가

파라미터가 더 많다 → 최소한 외우기는 더 잘해야 함

그런데 **외우는 것부터 더 못함**

용량이 모자라서 생긴 일이 아님

확인해 보기

층을 더 넣되

그 층이 아무것도 안 하면

어떻게 되는가?

논리적으로는 나빠질 이유가 없다

층을 더 넣되, 넣은 층이 아무것도 안 하게 만들면?

① 얇은 모델



기준

② 층을 더하되 통과만 시킴



①과 결과가 같아야 함

③ 실제로 conv 층을 더함



학습해 보면 ①보다 나빠짐

②가 가능하다면 ③도 최소한 ①만큼은 되어야 함. 그런데 관찰은 그렇지 않음
즉 더 깊은 모델이 얇은 모델을 흉내내는 것부터 잘 안 된다는 뜻임

논리와 관찰이 어긋남

논리	깊은 쪽이 얇은 쪽보다 나쁠 이유가 없음
관찰	실제로는 더 나쁨

깊은 모델이 얇은 모델을 흉내내는 것부터 잘 안 된다

표현력이 아니라 **학습이 되느냐**의 문제임

자주 붙는 설명

"층이 깊어서 gradient가 사라진 것"

- 그럴듯하고, 실제로 관련 있는 현상임
- 그런데 이 관찰을 혼자서 설명하지는 못함
- 정규화 같은 장치를 넣어도 이 현상은 남음

본 것과, 그 이유는 다른 칸에 적는다

관찰한 것

깊게 만든 쪽이
train 손실부터 더 높다

학습 곡선에서 직접 읽을 수
있는 사실

자주 붙는 설명

“gradient가 사라져서다”

그렇듯하지만, 이 관찰을

혼자서 설명하지 못함

정규화를 넣어도 남는 현상임

말할 수 있는 것

깊이를 늘렸을 때
최적화가 더 어려워졌다

원인은 하나로 특정하지
않는다

원인을 못 정했다고 해서 대책을 못 세우는 것은 아님

“깊어질수록 학습이 어려워진다”는 관찰만으로도 다음 설계를 시도할 수 있음

오늘 말할 범위

말함 깊이를 늘렸더니 학습이 더 어려워졌음
말하지 않음 그 원인은 X 하나다

원인을 특정하지 못해도 **다음 설계는 시도할 수 있음**

| “ 관찰과 원인을 같은 문장에 넣지 않기

”

다음 질문

총에게 시키는 일을

바꿀 수는 없는가?

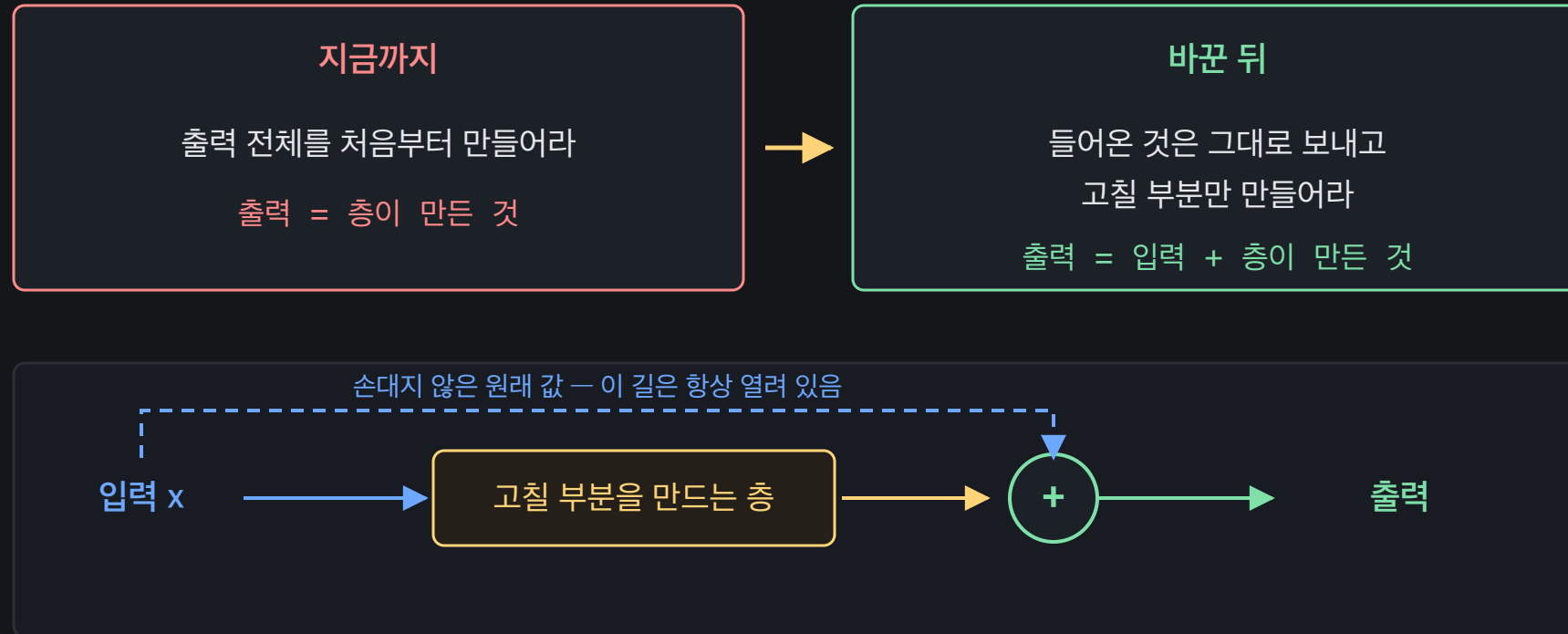
지금까지 층이 하던 일

출력 = 층이 처음부터 만든 것

"통과만 시켜라"도 층 입장에서는 **처음부터 만들어야 하는 일임**

가운데에 ReLU 같은 것이 끼어 있으면 더 그렇고

층에게 시키는 일을 바꾼다



층이 아무것도 안 하기로 하면, 만드는 값을 0에 가깝게 두면 됨

처음부터 통과값을 재현하는 것보다 이쪽이 다루기 쉬움

기존 정보는 그대로 보내고
필요한 수정만 더한다

왜 이쪽이 쉬운가

전	"입력과 똑같은 것을 만들어라"
후	"고칠 게 없으면 0을 만들어라"

- 0을 만드는 것은 가중치를 작게 두면 됨
- 통과값을 재현하는 것보다 다루기 쉬움

“ 쉽다는 것이 잘 된다는 보장은 아님

”

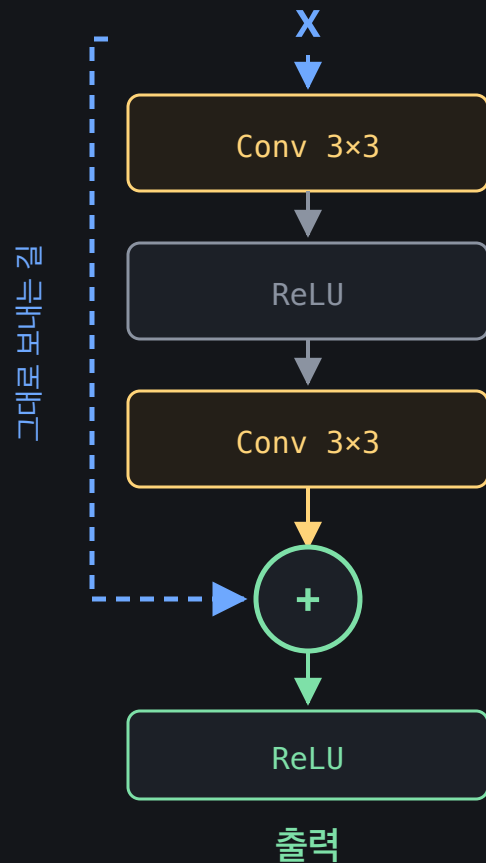
한 줄로 쓰면

출력 = 입력 + 총이 만든 것

여기서 **더하기 하나**가 오늘의 전부임

가장 단순한 형태로 만들어 본다

BatchNorm 없음 · 채널과 크기가 그대로인 경우



```
class MinimalResidual(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.c1 = nn.Conv2d(ch, ch, 3, padding=1)
        self.c2 = nn.Conv2d(ch, ch, 3, padding=1)

    def forward(self, x):
        out = torch.relu(self.c1(x))
        out = self.c2(out)
        return torch.relu(out + x)
```

중요한 줄은 하나 — `out + x`

채널도 크기도 안 변했기 때문에 그냥 더해짐

파라미터 73,856개 · 입력 (2,64,16,16) → 출력 (2,64,16,16)

코드에서 볼 것

```
def forward(self, x):  
    out = torch.relu(self.c1(x))  
    out = self.c2(out)  
    return torch.relu(out + x)
```

- `out + x` — 이 한 줄이 shortcut임
- `x` 는 아무 층도 거치지 않고 그대로 옴
- BatchNorm은 넣지 않았음. 개념을 먼저 보기 위함

한 번 통과시켜 보면

입력	(2, 64, 16, 16)
출력	(2, 64, 16, 16)
파라미터	73,856개

채널도 크기도 안 변했음 → 그래서 **그냥 더해짐**

만드는 값을 0으로 두면

두 번째 conv의 가중치를 전부 0으로 만들고 통과시킴

```
출력 == ReLU(입력)    True
```

층이 **아무것도 안 하기**를 고를 수 있음이 확인됨

“ 학습이 실제로 이걸 고른다는 뜻은 아님. 고를 수 있다는 뜻임 ”

자주 듣는 설명 하나 더

"미분하면 최소 10이라

안전하다"

맞는 말인가?

“미분하면 최소 1이라 안전하다”는 말은 맞는가

$$F(x) + x \text{ 를 } x \text{로 미분하면 } dF/dx + 1$$

그러니까 항상 1 이상이다 — 라고 흔히 말함

$$F(x) = 2x \text{ 라면}$$

$$2 + 1 = 3$$

1보다 큼

$$F(x) = -x \text{ 라면}$$

$$-1 + 1 = 0$$

0이 됨

$$F(x) = -2x \text{ 라면}$$

$$-2 + 1 = -1$$

음수가 됨

dF/dx 는 음수가 될 수 있으므로 1 이상은 보장되지 않음

말할 수 있는 것은 여기까지 — 더하기 때문에 gradient 향이 하나 늘어난다

향이 하나 더 생기는 것과, 그 합이 항상 크다는 것은 다른 말임

정정

주장	$dF/dx + 1$ 은 항상 1 이상
사실	dF/dx 가 음수면 0도 되고 음수도 됨

말할 수 있는 것은 여기까지

더하기 때문에 gradient가 흐르는 길이 하나 더 생김

이 한 줄을 넣고 무엇이 달라졌는가

말할 수 있는 것

- 깊게 쌓아도 학습이 시작은 됨
- 층이 “아무것도 안 하기”를 고를 수 있음
- 더하기 때문에 gradient 경로가 하나 늘어남

말할 수 없는 것

- × 정확도가 반드시 올라간다
- × 깊을수록 항상 좋아진다
- × gradient가 1 이상으로 유지된다
- × overfitting이 줄어든다

이 블록에서 확인한 것은 구조와 shape뿐임. 학습 결과는 여기서 재지 않았음

다음 질문 — 블록 하나로는 네트워크가 안 됨. 이것 어떻게 이어 붙이는가

오늘 정리

진단	train·dev 둘을 같이 봐야 이름이 붙음
degradation	깊게 했는데 train부터 나빠짐
논리	통과만 시켜도 되니 나빠질 이유가 없음
처방	그대로 보내는 길을 하나 뚫
코드	<code>out + x</code> 한 줄

남는 문제

블록 하나는 만들었음

```
(2, 64, 16, 16) → (2, 64, 16, 16)
```

채널을 늘리고 크기를 줄이려면 어떻게 하는가

블록 하나가 어떻게

전체 네트워크가 되는가?

블록에서 네트워크로

CNN · 블록 16

앞 블록에서 만든 것

```
return torch.relu(out + x)
```

입력 (2, 64, 16, 16) → 출력 (2, 64, 16, 16)

채널도 크기도 그대로였기 때문에 그냥 더해졌음

오늘의 질문

블록 하나가 어떻게

전체 네트워크가 되는가?

네트워크가 되려면 필요한 것

지금까지 본 모든 구조가 하던 일

뒤로 갈수록 채널은 늘고, 공간은 줄어든다

블록을 그냥 이어 붙이면 이게 안 됨

어딘가에서 채널을 늘리고 크기를 줄여야 함

그럼 그냥 해 보기

```
conv_path = nn.Sequential(  
    nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.ReLU(),  
    nn.Conv2d(128, 128, 3, padding=1),  
)  
out = conv_path(x)  
y = out + x
```

무슨 일이 생길까?

채널을 늘리고 크기를 줄이는 순간, 더하기가 깨진다



`RuntimeError: The size of tensor a (16) must match the size of tensor b (32) at non-singleton dimension 3`
채널도 다르고 크기도 다름 — 자리끼리 짝이 안 이어짐

오류를 읽기

```
RuntimeError: The size of tensor a (16) must match  
the size of tensor b (32) at non-singleton dimension 3
```

- 마지막 축이 16인데 상대는 32라고 말해 줌
- 채널도 128 대 64로 다름
- 더하기는 **자리마다 짝을 지어야** 하는 연산임

그러면

무엇을 고쳐야 하는가?

- 층 쪽을 포기한다?
- 아니면 그대로 보내는 쪽을 손본다?

필요한 도구를 이미 배웠음

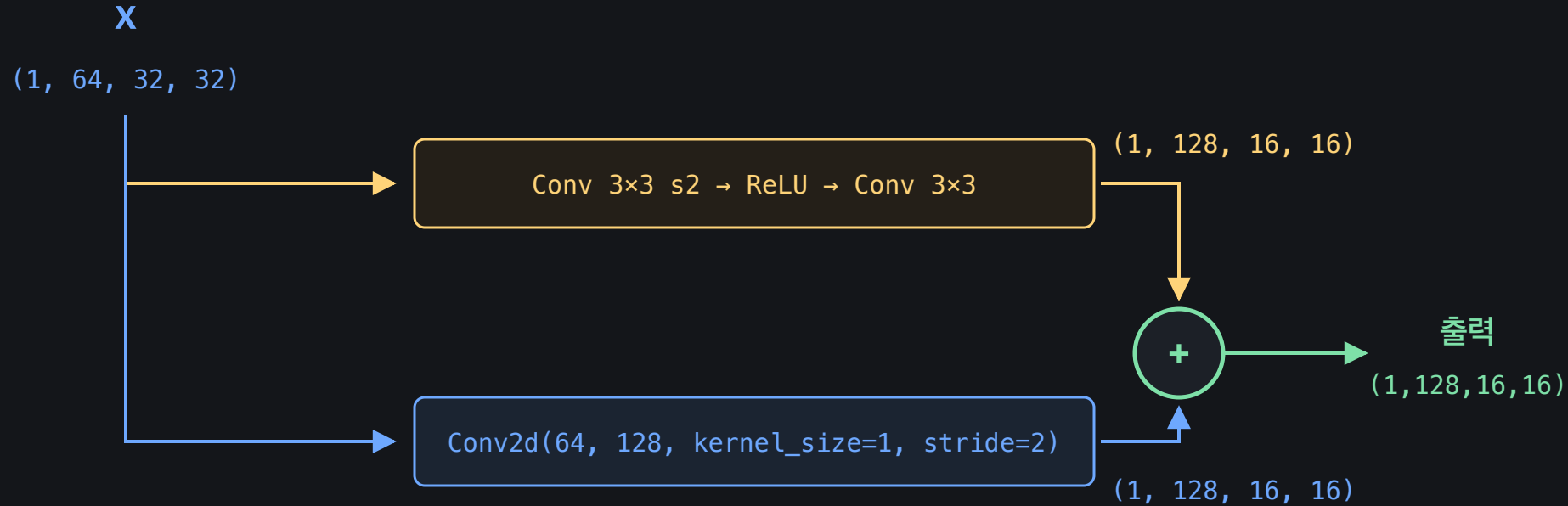
자리는 그대로 두고, 채널만 바꾸는 도구

1×1 Convolution

크기까지 줄이려면 `stride` 를 함께 주면 됨

shortcut 쪽 shape을 맞춰 주면 된다

블록 14에서 본 1×1 Convolution



채널을 64→128로 1×1이 바꾸고, stride 2가 크기를 32→16으로 맞춤
새 장치가 아님. 블록 14에서 “자리는 두고 채널만 바꾸는 도구”라고 부른 그것임
이 1×1이 쓰는 파라미터 8,320개 — 같은 일을 3×3으로 하면 73,856개

한 줄 추가

```
proj = nn.Conv2d(64, 128, kernel_size=1, stride=2)
```

```
identity = proj(x)          # (1, 128, 16, 16)
```

```
y = out + identity          # 더해짐
```

proj 파라미터	8,320개
-----------	--------

같은 일을 3×3로 하면	73,856개
---------------	---------

이걸 부르는 이름

projection shortcut

- 새로 발명한 장치가 아님
- 블록 14에서 쓰던 1×1 을 shortcut 쪽에 놓은 것
- 목적은 하나 — **더할 수 있게 shape을 맞추는 것**

“ shape을 맞추려고 넣은 층임. 성능을 위해 넣은 층이 아님

”

다음 질문

이 블록들을

어떻게 묶는가?

블록을 몇 개 이어 붙인 묶음을 stage라고 부른다

크기를 줄이는 일은 첫 블록에서 한 번만 한다



stage 안에서 shape이 바뀌는 지점은 딱 한 곳 — 첫 블록임
그래서 1x1 projection도 첫 블록에만 붙음. 나머지는 그냥 더하면 됨
블록을 몇 개 넣을지는 설계 선택임. 정해진 답이 있는 값이 아님

stage 규칙

위치	stride	shortcut
첫 블록	2	1×1 projection
나머지	1	그냥 통과

shape이 바뀌는 지점은 stage마다 **한 곳**뿐임

stage를 만드는 코드

```
def make_stage(cin, cout, stride, n_blocks):  
    layers = [Block(cin, cout, stride)]  
    for _ in range(n_blocks - 1):  
        layers.append(Block(cout, cout, stride=1))  
    return nn.Sequential(*layers)
```

- 첫 블록만 `stride` 를 받고, 나머지는 1
- 첫 블록만 `cin → cout`, 나머지는 `cout → cout`
- 읽기만 함. 지금 고치지 않음

부르는 쪽

```
stage1 = make_stage(64, 64, stride=1, n_blocks=2)
stage2 = make_stage(64, 128, stride=2, n_blocks=2)
stage3 = make_stage(128, 256, stride=2, n_blocks=2)
```

숫자만 보면 규칙이 보임

채널	64	→	64	→	128	→	256
stride			1		2		2

통과시켜 보기 전에

각 stage를 나오면

shape이 어떻게 될까?

입력은 (4, 3, 32, 32)

블록이 모여 네트워크가 되면 무엇이 남는가

CIFAR-10 배치 4장 통과 · torch 실측

	나온 shape	첫 블록	파라미터
입력	(4, 3, 32, 32)	—	—
첫 conv	(4, 64, 32, 32)	—	1,792
stage 1 · 블록 2개	(4, 64, 32, 32)	stride 1	147,712
stage 2 · 블록 2개	(4, 128, 16, 16)	stride 2	524,928
stage 3 · 블록 2개	(4, 256, 8, 8)	stride 2	2,098,432
GAP + Linear	(4, 10)	—	2,570

stage를 지날 때마다 채널은 2배, 한 변은 절반

총 2,775,434개 · 마지막에 GAP이 8×8을 값 하나로 눌러 Linear에 넘김

표에서 읽을 것

공간	32 → 32 → 16 → 8
채널	64 → 64 → 128 → 256

- stage 안에서는 안 변하고, **stage가 바뀔 때** 변함
- 파라미터는 뒤쪽 stage에 몰려 있음 — 채널이 커졌기 때문

마지막 처리

$(4, 256, 8, 8) \rightarrow \text{GAP} \rightarrow (4, 256) \rightarrow \text{Linear} \rightarrow (4, 10)$

- 채널마다 8×8 을 평균 내어 값 하나로
- flatten보다 Linear의 입력이 훨씬 작아짐
- 출력은 raw logits — 손실 함수가 이어받음

실제 구현을 열어 보면

```
nn.Conv2d(...), nn.BatchNorm2d(...), nn.ReLU()
```

BatchNorm2d 가 conv 뒤마다 붙어 있음

학습을 안정시키는 장치임. 오늘은 다루지 않음

구조를 읽을 때는 conv와 더하기만 따라가면 됨

오늘 하지 않은 것

- 실제로 학습시키지 않았음
- 정확도를 재지 않았음

확인한 것은 **shape이 맞아떨어지는지**까지임

“ 구조가 돌아간다는 것과 잘 배운다는 것은 다른 말임 ”

이들을 되감으면

네 모델은 각각

무슨 질문에 답한 것인가?

네 모델이 각각 답한 질문



뒤 모델이 앞 모델을 이겼다는 이야기가 아님. 각자 다른 문제를 다뤘음
어떤 구조가 좋은지는 데이터·계산 예산·평가 방식이 정해져야 답할 수 있는 질문임

흐름을 읽는 방식

X	뒤 모델이 앞 모델을 이겼다
0	각자 다른 문제를 하나씩 다뤘다

어떤 구조가 나온지는

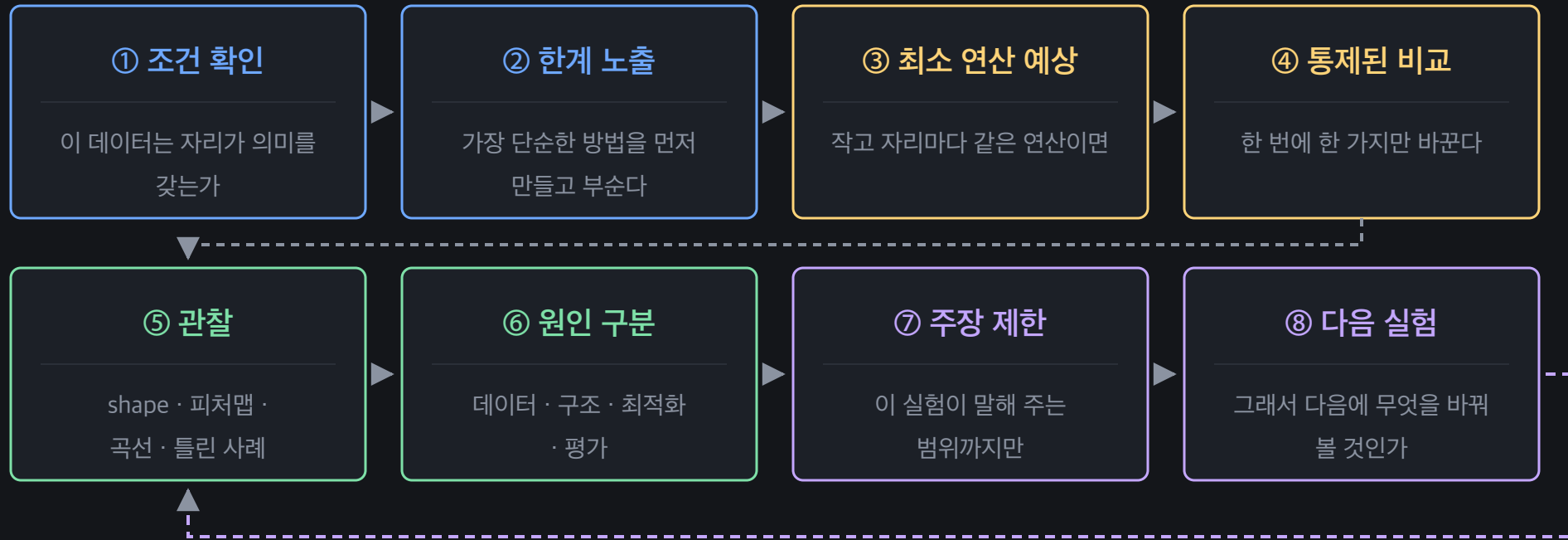
데이터 · 계산 예산 · 평가 방식이 정해져야 답할 수 있음

이틀 동안 반복한 순서

조건 확인 → 가장 단순한 방법의 한계 노출
→ 가장 작은 연산을 예상 → 한 번에 하나만 바꿔 비교
→ shape · 피처맵 · 학습 곡선 · 틀린 사례 관찰
→ 데이터 / 구조 / 최적화 / 평가 중 어디인지 구분
→ 주장 범위 제한 → 다음 실험 선택

이들 동안 반복한 순서

모델 이름이 아니라, 이 순서가 남는 것



⑧에서 다시 ①로 — 끝나는 목록이 아니라 도는 순서임

남는 것은 모델 이름이 아니라
이 순서임

이틀 지도

이미지	값 + 자리 · (N, C, H, W)
합성곱	작은 창 · 자리마다 같은 가중치
크기	stride · padding · pool
학습	train / dev · 손실 · 옵티마이저
구조	3×3 반복 · 1×1 · shortcut · stage

스스로 점검하기

각 항목을 **옆 사람에게 설명할 수 있는지**로 확인

1. 합성곱과 풀링이 무엇을 하는지 말할 수 있는가
2. 왜 이미지에 합성곱을 쓰는지 말할 수 있는가
3. LeNet · AlexNet · VGG · ResNet이 각각 무슨 문제를 다뤘는지 말할 수 있는가
4. shortcut이 왜 필요했는지 말할 수 있는가
5. 코드에서 합성곱·풀링 구조를 짚을 수 있는가

마지막으로

막히면 돌아올 자리

1. 이 데이터는 자리가 의미를 갖는가
2. 가장 단순한 방법으로 먼저 해 보면 어디서 깨지는가
3. 무엇을 하나만 바꿔서 비교할 것인가
4. 그래서 무엇까지 말할 수 있는가

이 네 줄이 오늘의 결론임